

VGGT: Visual Geometry Grounded Transformer

발표자 : 이진주

자료조사 : 안가영, 안종식, 최비결, 황장훈

Introduction

➡ 연구 배경 및 동기

기존 3D 재구성은 기하 기반 최적화(예: BA)에 의존 → 계산 복잡도 높음

최신 SfM 기법(VGGSfM)은 미분 가능한 BA로 엔드-투-엔드 학습을 시도하지만, 여전히 후처리 필요

최근 DUSfM, MASfM는 피드포워드 방식으로 가능성을 보였지만 이미지 쌍(pair)에만 제한

➡ 연구 목표

후처리 없는 완전한 피드포워드 방식으로, 여러 이미지에서 3D 속성을 직접 추정하는 모델 설계

Introduction

▶ 제안 기법 : VGGT

여러 장의 이미지 입력으로 카메라 파라미터, 데프스 맵, 3D 포인트 맵, 포인트 트래킹을 **단일 forward pass로 예측**
후처리 없이도 기존 방법보다 우수한 정확도

범용 트랜스포머 구조 (특수한 3D inductive bias 없음)

다양한 3D 작업(시점 생성, 트래킹 등)에 재활용 가능

▶ 성능 비교 및 특징점

DepthAnything, LRM 등 단일 작업 특화 모델들과 달리,

→ VGGT는 3D 전체 속성을 통합 예측하는 단일 백본

관련된 3D 속성을 동시 학습하면 전반적인 성능 향상

추론 시, 개별적으로 예측한 depth + camera로 추론한 point map이 오히려 전용 head보다 정확함

Introduction

➡ 기여 요약

다중 이미지 기반 3D 추론이 가능한 대형 트랜스포머 모델(VGGT) 제안
후처리 없는 방식으로 기존 SOTA 기법보다 빠르고 정확함
BA와 결합 시, 모든 항목에서 최고 성능 달성

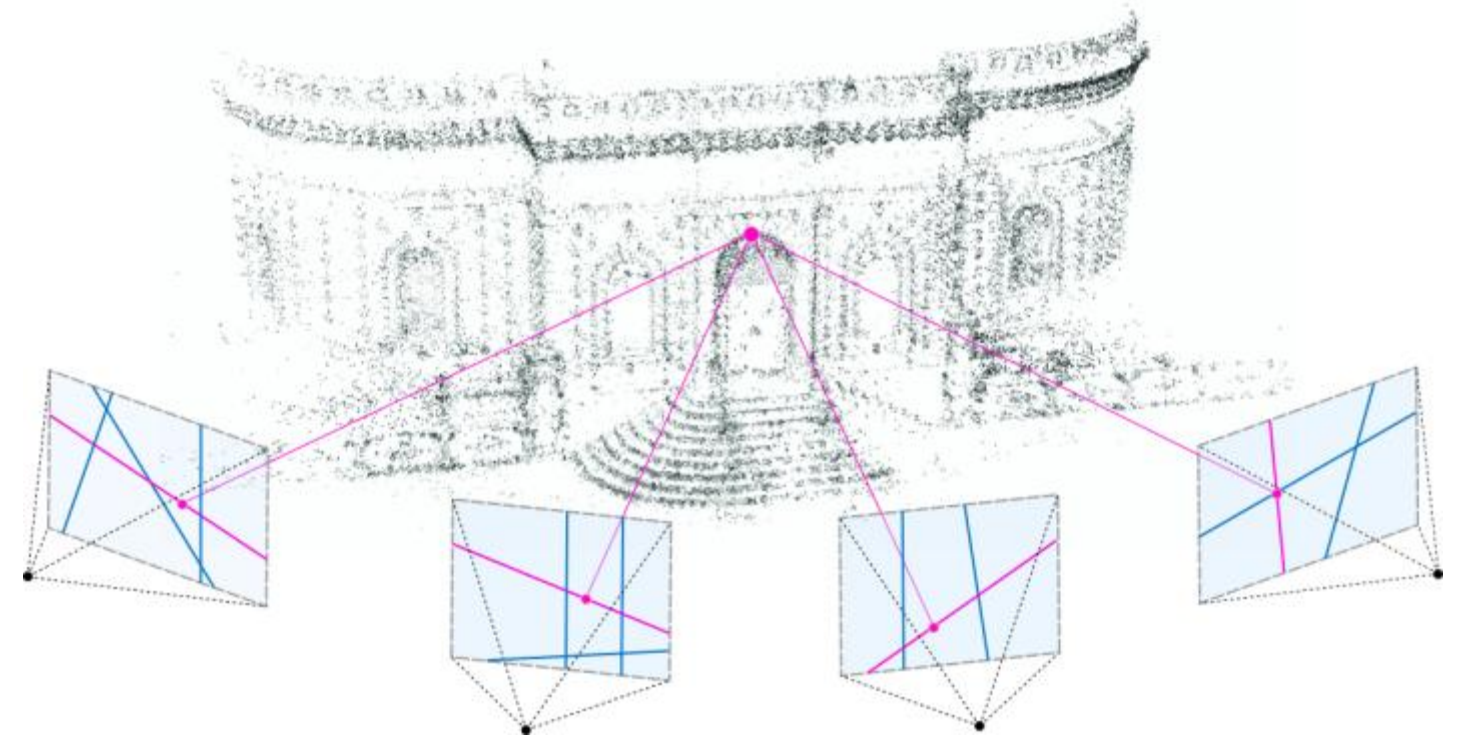
Related Work

➡ Structure from Motion (SfM)

정의 : 2D 이미지로부터 3D 구조와 카메라의 위치를 복원하는 기술로,
여러 장의 이미지만으로 장면의 3차원 구조를 재구성 하는 문제

기존

COLMAP(SfM과 MVS를 지원 라이브러리)과 같은
이미지 매칭 → 삼각측량 → 번들 조정의
다단계 절차로 구성된 파이프라인 사용



최근

딥러닝이 SfM 파이프라인의 여러 요소
(키포인트 검출, 이미지 매칭)를 향상시킴
VGGSfM과 같은 최신 SfM 방법들은 differentiable BA를 통해
머신 러닝과 비주얼 지오메트리를 엔드-투-엔드로 결합하여
전통적인 알고리즘을 능가하기 시작

Related Work

► Multi-view Stereo (MVS)

정의: 주로 SfM으로 추정된 카메라 파라미터를 가정하고, 여러 겹치는 이미지로부터 3D장면의 기하를 조밀하게 복원하는 문제
categories : traditional handcrafted, global optimization, learning-based methods

최근 SfM과 마찬가지로 **learning-based method**가 급속히 발전하고 있음

ex) 카메라 파라미터 없이, 두 장의 이미지만으로 정렬된 dense point cloud 직접 추정하는 DUS3R와 MAST3R

-> 여전히 쌍(pair) 단위로만 동작하며, 장면 전체를 복원하려면 후처리 병합 및 정렬이 필요하다는 한계 존재

반면, **VGGT**는 **DUS3R** 및 **MAST3R**보다 훨씬 뛰어난 성능을 나타냄



Related Work

▶ Tracking-Any-Point

정의: 비디오에서 관심 점의 위치를 프레임 전체에 걸쳐 추적하는 문제

(비디오와 일부 2D점들이 주어지면, 모든 다른 프레임에서 해당 점들의 2D 대응점을 예측)

주요 기법: TAP-Vid, TAPIR, CoTracker, DOT, TAPTR, LocoTrack 등 다양한 딥러닝 기반 point tracker들이 존재.

TAP-Vid : 관련된 세 가지 벤치마크를 제안하고, 단순한 기준 모델을 발표.이후 TAPIR에서 성능을 향상시킴

CoTracker : 여러 점 사이의 상호 연관을 활용해 가림(occlusion) 상황에서도 추적

DOT : occlusion을 뚫고 dense 추적을 시도

TAPTR : 엔드투엔드 트랜스포머 기반

LocoTrack : 점 단위 특징을 주변 영역까지 확장

-> **VGGT**과 기존 **point tracker**를 결합할시 **최고 수준의 추적 성능을 달성가능**

Method

► Problem definition and notation

입력: 동일한 3D 장면을 여러시점에서 촬영한 N개의 RGB 이미지 시퀀스 ($I_0, I_1 \sim I_N$)

출력: 각 이미지에 대해 다음의 3D 정보 예측

- g : Camera Params (9개의 실수)
- D : Depth Map (1차원 HxW)
- P : Point Map (3차원 HxW)
- T : Point Tracking을 위한 Feature Map (CxHxW)

$$f \left((I_i)_{i=1}^N \right) = (g_i, D_i, P_i, T_i)_{i=1}^N .$$

Method

➡ Problem definition and notation

Camera Params(G_i) :

- Camera Params은 VGGSfM에서 제안한 Parameterization을 사용
- $g=[q,t,f]$ 는 **Rotation Quaternion q (4개의 실수), translation Vector t (3개의 실수), Field of View f (2개의 실수)** 로 구성
 - Rotation Quaternion q : 카메라의 3D 공간에서의 회전(rotation)을 나타내는 쿼터니언(quaternion).
 - translation Vector t : 카메라의 3D 공간에서의 변환(translation), 즉 위치를 나타내는 벡터
 - Field of View f : 카메라의 시야각(field of view). 이미지 센서 크기, 초점 거리 등 카메라의 내부 광학적 특성과 관련
 - 카메라의 Principal Point는 이미지 중심이라 가정하고, 이는 SfM Framework에서 일반적으로 사용되는 방법

Depth Map(D_i) :

- 각 이미지 I_i 의 픽셀 위치 y 에 대해 해당 픽셀에 해당하는 양의 실수 깊이값 $D_i(y)$ 예측

Method

➡ Problem definition and notation

Point maps (P_i):

- 각 이미지 I_i 의 픽셀 위치 y 에 대해 해당 픽셀에 해당하는 3D 장면 좌표
- DUS3R처럼, 포인트 맵은 viewpoint invariant하며,
⇒ 즉, 3D 포인트는 첫 번째 카메라 g_1 의 좌표계를 월드 좌표계로 사용하여 정의

keypoint tracking(T_i):

- track-any-point 방법을 따름
⇒ 즉, 쿼리 이미지 I_q 의 쿼리 점 y_q 가 주어지면, 모든 다른 이미지 I_i 에서의 2D 대응점들 y_i 로 구성된 트랙(y_q)을 예측
- 트래킹 모듈 T 의 입력으로 사용되는 특징 맵을 출력(실제 추적 작업은 별도의 모듈에 위임됨)
- 트랜스포머 f 와 트래킹 모듈 T 는 함께 학습됨

$$T \left((y_j)_{j=1}^M, (T_i)_{i=1}^N \right) = \left((\hat{y}_{j,i})_{i=1}^N \right)_{j=1}^M$$

Method

► Problem definition and notation

Order of Predictions:

- 입력 이미지 시퀀스의 순서는 임의적이며, 단 첫 번째 이미지만 기준(reference) 프레임으로 지정.
- 네트워크 구조는 첫 번째 프레임을 제외한 나머지 프레임들에 대해서는 순서에 상관없이 동일한 결과를 내도록 설계(permutation equivariant)

Over-complete Predictions:

- VGGT가 예측하는 모든 항목들이 서로 독립적이지 않음
- DUS3R에서 처럼, invariant point map P 로부터 PnP(Perspective-n-Point)와 같은 방법을 통해 카메라 파라미터 g 추정 가능
- 포인트 맵과 카메라 파라미터로부터 깊이 맵을 도출 가능
- 그러나 모든 항목을 명시적으로 Prediction하도록 Training 하는게 성능 향상에 도움이 되는걸 실험적으로 확인함
- 또한, 추론시 독립적으로 예측된 Depth Map과 Camera Parmas를 결합하여 Point Map을 계산하는게 Head를 통해 Prediction하는것 보다 더 정확한 결과임을 확인함.

Method

Feature Backbone

최소한의 3D 귀납적 편향(inductive bias)을 가진 단순한 아키텍처로 설계

- 대형 트랜스포머(large transformer)로 설계
- 충분한 양의 3D 주석 데이터로부터 모델이 직접 학습시킴

Process:

1. DINO를 사용해 입력 이미지를 패치화하고, 토큰화
2. 모든 프레임으로부터 얻은 토큰 집합을 프레임 단위와 전체(global) 단위의 셀프 어텐션 층을 교대로 적용하는 메인 네트워크 구조(Alternating Attention)에서 처리

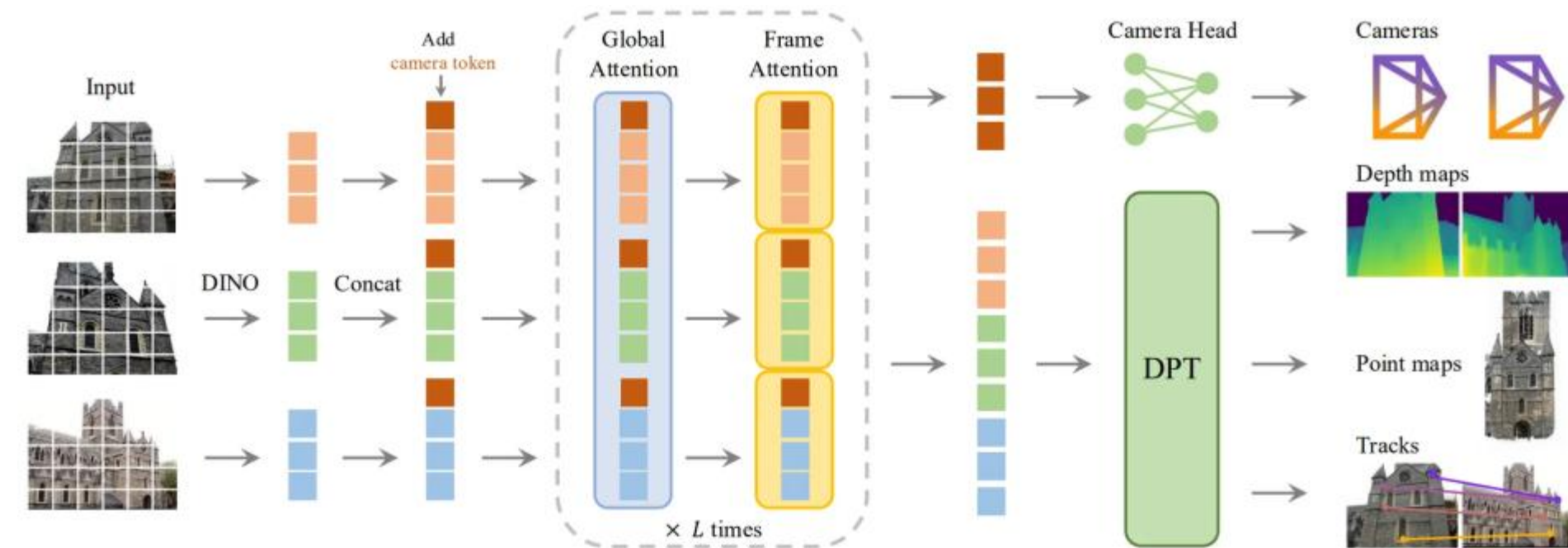


Figure 2. **Architecture Overview.** Our model first patchifies the input images into tokens by DINO, and appends camera tokens for camera prediction. It then alternates between frame-wise and global self attention layers. A camera head makes the final prediction for camera extrinsics and intrinsics, and a DPT [87] head for any dense output.

Method

► Feature Backbone

Alternating-Attention :

AA: 기존의 **transformer self-attention** 을 변형

트랜스포머가 각 프레임 내부와 프레임 전체에 걸쳐 번갈아(attention alternately) 주의를 기울이도록 함.

Frame-wise self-attention:

각 프레임 내의 token에 대해 **개별적으로 attention**을 수행하여 해당 이미지 내의 정보 통합에 집중

Global self-attention:

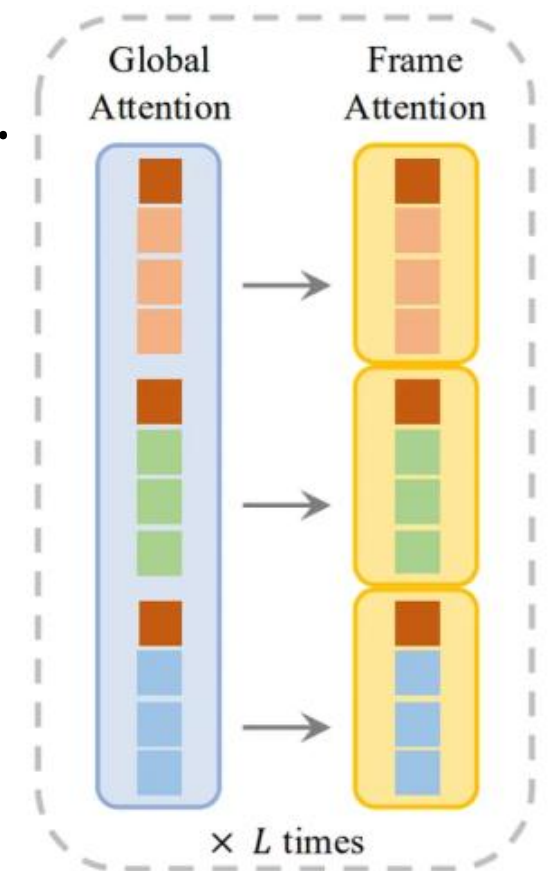
모든 프레임의 token에 대해 **함께 attention**을 수행하여 이미지 간의 관계 및 global 정보를 통합

기본적으로 L=24개의 레이어에 대해 Frame-wise & global self-attention 적용.

-> 이러한 구조는 서로 다른 이미지 간의 정보를 통합하면서도, 각 이미지 내부의 토큰들에 대해

정규화(normalization) 효과를 부여하는 균형잡힌 구조를 형성

해당 아키텍처는 크로스 어텐션(cross-attention)은 전혀 사용하지 않고, 오직 셀프 어텐션 만을 사용



Method

► Prediction heads

토큰 구성 및 입력 처리:

입력 이미지 I_i 에 대해 해당 이미지 토큰 t_{I_i} 에 추가적으로 아래 토큰을 더함

- 하나의 카메라 토큰
- 네개의 Register 토큰
- 위 모든 토큰을 Concat 해서 AA Transformer에 전달되고 출력 토큰 생성됨

$$(t_i^I, t_i^g, t_i^R)_i^N$$

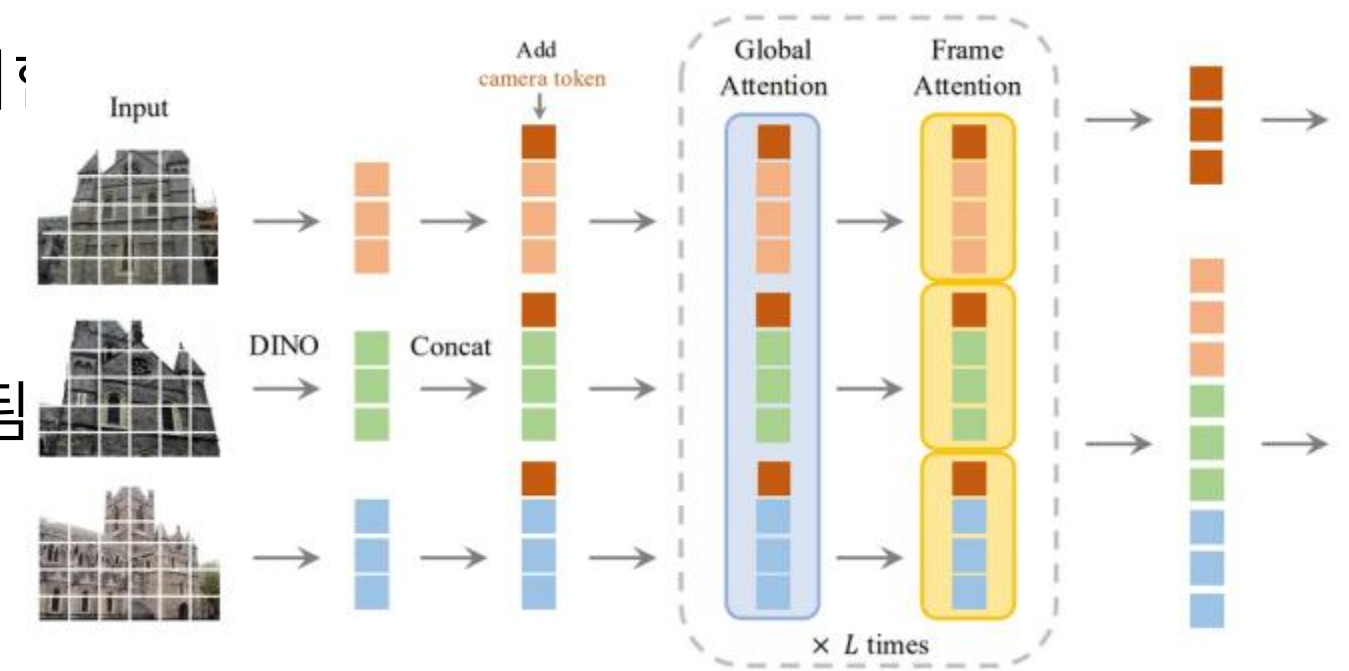
Concat한 토큰 (입력)

$$(\hat{t}_i^I, \hat{t}_i^g, \hat{t}_i^R)_i^N$$

출력 토큰

첫 프레임의 카메라/레지스터 토큰은 별도의 학습 가능한 토큰으로 설정:

→ 첫 프레임을 기준 좌표계로 인식하게 함



$$t_1^g := \bar{t}^g, t_1^R := \bar{t}^R$$

Method

► Prediction heads

Coordinate Frame:

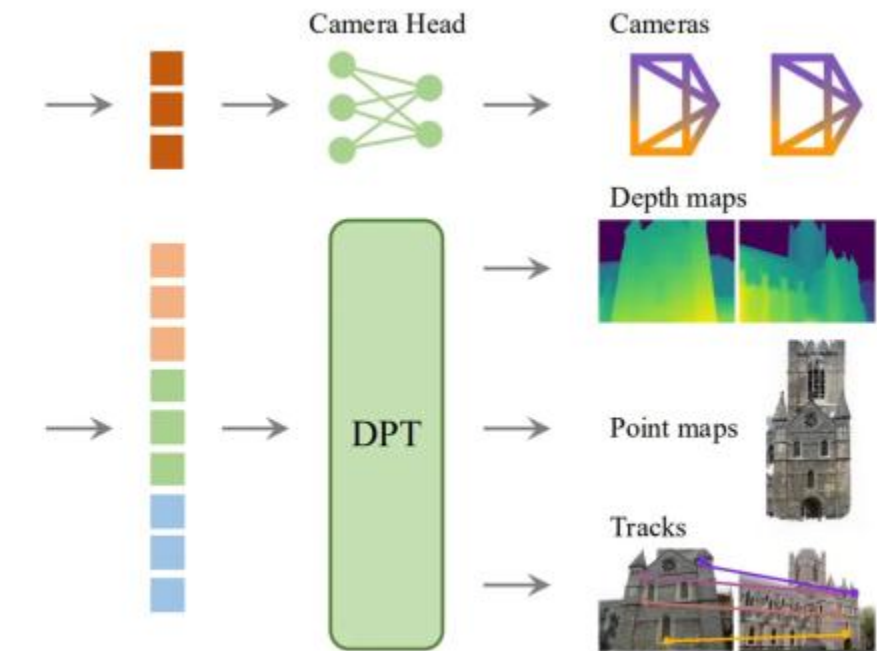
모든 Camera, Point Map, Depth Map은 첫번째 카메라 g_i 의 좌표계를 따름
때문에, 첫번째 카메라의 Extrinsic Parameter는 $q_1=[0,0,0,1]$, $t_i=[0,0,0]$
이러한 설정은 첫 프레임을 명확히 구분하는 토큰으로 가능

Camera Predictions:

카메라 내부/외부 파라미터를 예측

카메라 파라미터는 출력카메라 토큰으로 부터 생성됨

이 과정에는 4개의 Self Attention Layer와 하나의 Linear Layer로 구성되어 있고 이를 Camera Head라고 부름.



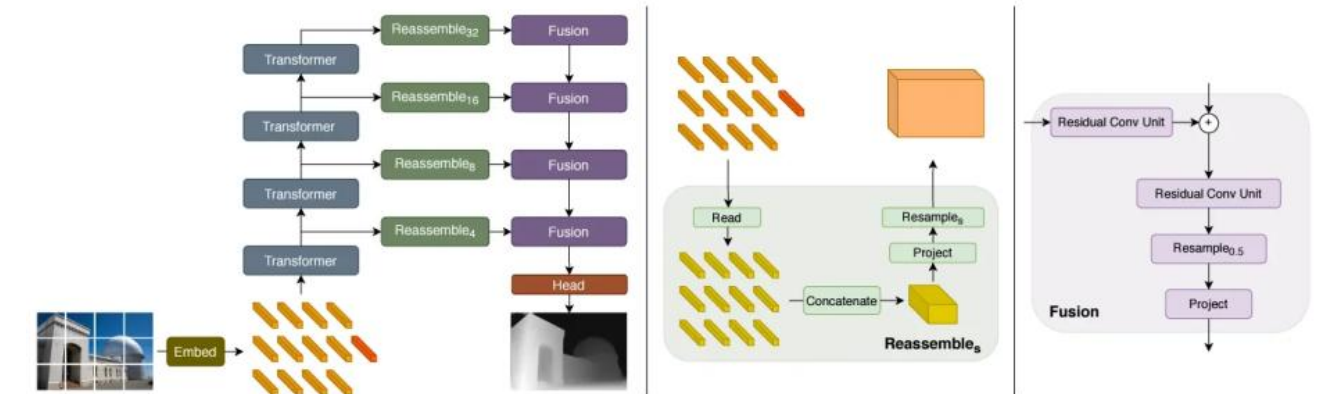
► Prediction heads

Dense Predictions:

출력 이미지 토큰은 먼저 DPT레이어를 통해 dense 특징 맵으로 변환됨
이후 3×3 컨볼루션을 통해, depth 맵, point 맵 생성하며 각각에 대한

Aleatoric Uncertainty도 함께 예측

이 불확실성 맵은 학습 시 손실 함수에 사용되며, 학습 후에는 모델의 예측 신뢰도를 나타냄



Tracking:

Tracker T는 CoTracker2 아키텍처를 사용

1. bilinear sampling으로 쿼리 포인트에 대한 feature를 추출
2. 이 특징을 다른 프레임들의 특징 맵과 상관 계산(correlation)
3. 생성된 상관 맵들을 self-attention 레이어에 통과시켜 최종적으로 대응점예측

이 방식은 VGGsfM처럼 입력 프레임 간 시간 순서를 가정하지 않으며, 비디오가 아닌 임의의 이미지 집합에도 적용 가능

$$\mathcal{T}((\mathbf{y}_j)_{j=1}^M, (T_i)_{i=1}^N) = ((\hat{\mathbf{y}}_{j,i})_{i=1}^N)_{j=1}^M$$

Method

► Training

Training Losses:

엔드투엔드 방식으로 다음과 같은 멀티태스크 손실 함수를 사용해 학습
카메라, 깊이, 포인트맵 손실 항목들은 비슷한 범위를 가지므로 별도 가중치 없이 합산
트래킹 손실은 상대적으로 작게 반영 ($\lambda=0.05$)

$$\mathcal{L} = \mathcal{L}_{\text{camera}} + \mathcal{L}_{\text{depth}} + \mathcal{L}_{\text{pmap}} + \lambda \mathcal{L}_{\text{track}}$$

Camera Losses:

예측된 카메라 파라미터와 GT간의 Huber 손실

$$\mathcal{L}_{\text{camera}}^{\text{Huber}} = \sum_{i=1}^N \|\hat{\mathbf{g}}_i - \mathbf{g}_i\|_{\epsilon}$$

Method

► Training

depth Losses:

DUSt3R의 aleatoric 불확실성 기반 손실을 따름

추가적으로 깊이 gradient 기반 항목도 포함 (monocular depth estimation에서 일반적)

- Point Map도 동일하지만, Point map으로 변경해서 계산됨.

$$\mathcal{L}_{\text{depth}} = \sum_{i=1}^N \|\Sigma_i^D \odot (\hat{D}_i - D_i)\| + \|\Sigma_i^D \odot (\nabla \hat{D}_i - \nabla D_i)\| - \alpha \log \Sigma_i^D.$$

tracking Losses:

- 질의점에 대한 각 프레임에서의 GT 대응점과 예측점간의 거리 손실

$$\mathcal{L}_{\text{track}} = \sum_{j=1}^M \sum_{i=1}^N \|\mathbf{y}_{j,i} - \hat{\mathbf{y}}_{j,i}\|$$

- Cotracker2와 같이 포인트의 가시성 여부를 판단하는 BCE(이진 크로스엔트로피) 손실도 함께 적용

Method

► Training

Ground Truth Coordinate Normalization:

- 모호성을 제거하고 특정한 정규화된 표현을 Prediction하기 위해 Data를 정규화
 - 모든 정보를 첫번째 카메라 g_i 의 좌표계로 표현
 - Point Map에 포함된 모든 3D Point로부터 원점까지 평균 유클리디안 거리를 계산
 - 이 유클리디안 거리로 Camera Translation Vector t , Point Map P , Depth Map D 를 정규화
- 중요한점은 DUST3R과 달리 네트워크 결과에 대해 정규화를 안하고 훈련 데이터로부터 정규화를 직접 학습하도록 유도함.

Method

► Training

Implementation Details

- Default로, Global, Frame-wise Attention을 총 24개 Layer로 사용함, 전체 모델은 1.2B
- Optim : AdamW
- Iter : 160,000
- cosine learning rate Scheduler, Maximum learning Rate : 0.0002, warmup : 8,000 iters
- 각 mini batch에는 무작위로 선택된 Train Scene에서 2개~24개의 프레임을 Random Sampling
- 입력 프레임, Depth Map, Point Map은 518 Pixel로 Resize됨.
- Aspect Ratio : 0.33~1.0 으로 무작위 설정
- 입력 프레임에 대해 Color jitter, Gaussian Blur, Grayscale augmentation 적용
- A100 GPU 64개 사용하여 9일간 수행
- Gradient Clipping은 1.0의 임계값으로 적용
- bFloat16 Precision과 Gradient Check Pointing 사용

Method

► Training

Training Dataset

- Co3Dv2 ,BlendMVS, DL3DV, MegaDepth, Kubric, WildRGB, ScanNet, Hyper-Sim, Mapillary, Habitat, Replica, MVS-Synth, PointOdyssey, Virtual KITTI, Aria Synthetic Environments, Aria Digital Twin,
- 이 데이터셋들은 다양한 Domain, 합성데이터, 실제 데이터 모두 포함

Experiments

► Camera Pose Estimation

실험 개요:

데이터셋: CO3Dv2, RealEstate10K

지표: AUC@30 (RRA:Relative Rotation Accuracy + RTA: Relative Translation

Accuracy 기반 상대 자세 정확도)

-> AUC@30은 다양한 임계값에서 RRA와 RTA 중 최소값의 정확도를 누적인 면적

설정: 각 장면당 10개 이미지 샘플링, Co3Dv2에서만 학습

주요 성과 :

VGGT는 후처리 없이도 **Global Alignment(DUSt3R/MASt3R)**,

Bundle Adjustment(VGGSfM)기법 보다 더 높은 정확도를 달성

VGGT: 0.2초 (피드포워드), 타 기법들: 10초 이상 (후처리 포함)

Methods	Re10K (<i>unseen</i>) AUC@30 ↑	CO3Dv2 AUC@30 ↑	Time
Colmap+SPSG [92]	45.2	25.3	~ 15s
PixSfM [66]	49.4	30.1	> 20s
PoseDiff [124]	48.0	66.5	~ 7s
DUSt3R [129]	67.7	76.7	~ 7s
MASt3R [62]	76.4	81.8	~ 9s
VGGSfM v2 [125]	78.9	83.4	~ 10s
MV-DUSt3R [111] ‡	71.3	69.5	~ 0.6s
CUT3R [127] ‡	75.3	82.8	~ 0.6s
FLARE [156] ‡	78.8	83.3	~ 0.5s
Fast3R [141] ‡	72.7	82.5	~ 0.2s
Ours (Feed-Forward)	<u>85.3</u>	<u>88.2</u>	~ 0.2s
Ours (with BA)	93.5	91.8	~ 1.8s

Experiments

► Camera Pose Estimation

비교:

VGGT는 Fast3R등 빠른 방법들과 유사한 속도를 가지면서도, 동시대 학습 기반 방법들 보다는 성능이 우수

학습되지 않은 RealEstate10K 데이터셋에서도 VGGT가 압도적인 성능 우위를 보여, VGGT의 강력한 일반화 능력을 입증

VGGT 출력 결과를 BA로 미세 조정하면 정확도가 추가로 향상

Experiments

► Multi-view Depth Estimation

평가 데이터셋: DTU

지표: 표준 DTU 메트릭을 따름

Accuracy: 예측된 점에서 GT까지의 최소 유클리드 거리

Completeness: GT에서 예측된 점까지의 최소 유클리드 거리

Overall: Accuracy와 Completeness의 평균값 → Chamfer Distance

비교:

VGGT, DUS3R: GT 카메라 정보 없이 작동

MASt3R, GeoMVSNet 등: GT 카메라 사용

VGGT는 DUS3R 대비 Overall 점수를 1.741 → 0.382로 대폭 개선 하였으며, GT를 사용하는 방법들과 비슷한 성능 달성

-> VGGT가 가진 다중 이미지 학습 방식으로 인해 뛰어난 성능향상 가능(모델이 다중 시점 삼각측량 개념을 내재적으로 학습)

Known GT camera	Method	Acc.↓	Comp.↓	Overall↓
✓	Gipuma [40]	0.283	0.873	0.578
✓	MVSNet [144]	0.396	0.527	0.462
✓	CIDER [139]	0.417	0.437	0.427
✓	PatchmatchNet [121]	0.427	0.377	0.417
✓	MASt3R [62]	0.403	0.344	0.374
✓	GeoMVSNet [157]	0.331	0.259	0.295
✗	DUS3R [129]	2.677	0.805	1.741
✗	Ours	0.389	0.374	0.382

Experiments

► Point Map Estimation

데이터셋: ETH3D

평가지표: Accuracy, Completeness, Overall (Chamfer Distance)

정렬 방법: Umeyama 알고리즘 사용

입력: 장면당 10개 프레임, 마스크로 유효 포인트 필터링

비교:

VGGT는 DUS_t3R, MAS_t3R보다 훨씬 높은 정확도 달성

DUS_t3R/MAS_t3R는 최적화 후처리(10초) 필요

VGGT는 피드포워드 방식만으로 0.2초에 처리

깊이 + 카메라 예측 후 역투영한 포인트맵이 직접 예측한 포인트맵보다 더

정확

-> 학습은 세가지가 공동으로 이루어지더라도 복잡한 작업(포인트맵 예측)을

단순한 하위 과제(깊이 + 카메라 예측)로 분해하는 것이 성능에 도움이 된다는

것을 시사

Methods	Acc.↓	Comp.↓	Overall↓	Time
DUS _t 3R	1.167	0.842	1.005	~ 7s
MAS _t 3R	0.968	0.684	0.826	~ 9s
Ours (Point)	<u>0.901</u>	<u>0.518</u>	<u>0.709</u>	~ 0.2s
Ours (Depth + Cam)	0.873	0.482	0.677	~ 0.2s

Experiments

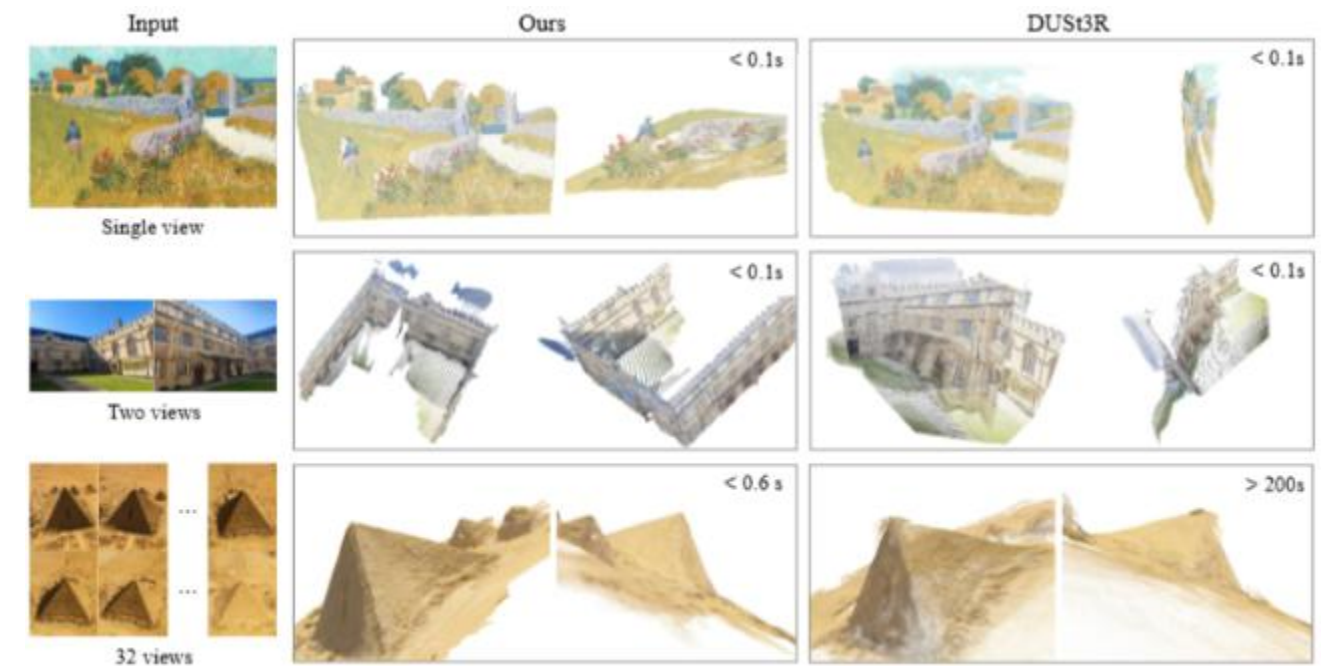
► Point Map Estimation

정성적 평가:

DUST3R와의 비교 시각화 예시

VGGT는 다음과 같은 까다로운 장면들에서도 고품질 예측 제공

- 유화 그림
- 프레임 겹침 없음
- 반복적/균일한 질감 (예: 사막)



Experiments

► Image Matching

VGGT는 두 뷰(view) 매칭에 특화된 모델은 아니지만, tracking branch의 정확도를 평가하기 위해 해당 태스크를 사용

데이터셋: ScanNet

평가 방식:

두 이미지에서 매칭점 추출 → Essential Matrix 추정 → 카메라 상대 자세 추정

AUC 기반 **relative pose accuracy**로 성능 평가

VGGT:

이미지 1에서 ALIKED로 키폰트 감지-> VGGT의 tracking branch를 이용해 이미지 2에서 대응점 찾기->

두 이미지 사이의 매칭 쌍들을 기반 Essential Matrix 추정 → 카메라 상대 자세 추정

Roma 논문의 하이퍼파라미터 (e.g. RANSAC) 사용

VGGT는 해당 태스크에 특화되지 않았음에도 불구하고, 기존 모든 베이스라인보다 높은 정확도 달성

-> 즉, 일반적인 3D 모델임에도 두 장의 이미지 간 정밀한 매칭 성능이 탁월함을 실험적으로 입증

Method	AUC@5 ↑	AUC@10 ↑	AUC@20 ↑
SuperGlue [92]	16.2	33.8	51.8
LoFTR [105]	22.1	40.8	57.6
DKM [32]	29.4	50.7	68.3
CasMTR [9]	27.1	47.0	64.4
Roma [33]	31.8	53.4	70.9
Ours	33.9	55.2	73.4

Experiments

► Ablation Studies

Feature Backbone:

목적 : 제안한 Alternating-Attention이 다른 attention 방식보다 효과적인지 검증

비교 대상 구조:

(a) Global Self-Attention만 사용하는 모델

(b) Cross-Attention 모델(각 프레임이 다른 모든 프레임의 토큰에 독립적으로 어텐션을 적용)

세 모델 모두 파라미터 수 및 하이퍼파라미터 동일하게 설정

평가 기준 : Point Map Estimation Accuracy (정확도)

Alternating-Attention이 가장 높은 성능 달성

Cross-Attention은 오히려 정확도 낮고 속도도 느림

ETH3D Dataset	Acc.↓	Comp.↓	Overall↓
Cross-Attention	1.287	0.835	1.061
Global Self-Attention Only	<u>1.032</u>	<u>0.621</u>	<u>0.827</u>
Alternating-Attention	0.901	0.518	0.709

Experiments

► Ablation Studies

Multi-task Learning:

목적 : 서로 중복될 수 있는 출력(예: 깊이 + 카메라 → 포인트맵)을 하나의 네트워크에서 함께 학습할 때의 효과 검증

VGGT는 여러 3D 속성(카메라, 깊이, 트랙 등)을 동시에 예측하도록 학습됨.

실험 결과:

특정 속성(예: 카메라, 깊이 등)을 제외하고 학습하면 포인트 맵 정확도 감

카메라 파라미터 학습이 정확도 향상에 가장 큰 기여

깊이는 상대적으로 기여도가 낮음

다중 3D 속성 학습이 성능 향상에 중요, 특히 카메라 파라미터 예측이 핵심 역할을 함

w. $\mathcal{L}_{\text{camera}}$	w. $\mathcal{L}_{\text{depth}}$	w. $\mathcal{L}_{\text{track}}$	Acc.↓	Comp.↓	Overall↓
✗	✓	✓	1.042	0.627	0.834
✓	✗	✓	<u>0.920</u>	<u>0.534</u>	<u>0.727</u>
✓	✓	✗	0.976	0.603	0.790
✓	✓	✓	0.901	0.518	0.709

Experiments

Finetuning for Downstream Tasks

Feed-forward Novel View Synthesis:

카메라 파라미터 없이도 새로운 시점의 이미지를 생성

LVSM 구조 기반으로 VGGT 간단 수정

→ 입력 이미지는 DINO로 토큰화, 타겟 시점은 Plücker ray로 표현

→ AA 트랜스포머 → DPT 헤드로 RGB 예측

적은 학습 데이터(Objaverse 20%)만 사용했음에도

→ GSO 데이터셋에서 LVSM과 유사한 성능 달성

카메라 정보 없이도 작동 가능하다는 점에서 높은 유연성과 잠재력을 입증

Method	Known Input Cam	Size	PSNR ↑	SSIM ↑	LPIPS ↓
LGM [110]	✓	256	21.44	0.832	0.122
GS-LRM [154]	✓	256	29.59	0.944	0.051
LVSM [53]	✓	256	31.71	0.957	0.027
Ours-NVS*	✗	224	30.41	0.949	0.033



Experiments

► Finetuning for Downstream Tasks

Dynamic Point Tracking:

CoTracker2의 백본을 VGGT 특징 추출기로 대체하여 실험

VGGT는 정적 이미지로 학습되었지만,

→ 예측된 tracking feature 를 CoTracker 구조에 적용

Kubric 데이터셋으로 파인튜닝 후 TAP-Vid벤치마크에서 성능 향상

→ 예: δ_{vis_avg} : 78.9 → 84.0

영상 기반 과제가 아님에도 우수한 일반화 성능 입증

즉, VGGT는 3D 재구성 외에도 시점 생성, 동적 추적 등 다양한 다운스트림 작업에 유연하게 활용 가능하며, 카메라 정보 없이도 높은 성능과 범용 표현력을 갖춘 모델임이 확인됨.

Method	Kinetics			RGB-S			DAVIS		
	AJ	$\delta_{vis_avg}^{vis}$	OA	AJ	$\delta_{vis_avg}^{vis}$	OA	AJ	$\delta_{vis_avg}^{vis}$	OA
TAPTR [63]	49.0	64.4	85.2	60.8	76.2	87.0	63.0	76.1	91.1
LocoTrack [13]	52.9	66.8	85.3	69.7	83.2	89.5	62.9	75.3	87.2
BootsTAPIR [26]	54.6	68.4	86.5	70.8	83.0	89.9	61.4	73.6	88.7
CoTracker [56]	49.6	64.3	83.3	67.4	78.9	85.2	61.8	76.1	88.3
CoTracker + Ours	57.2	69.0	88.9	72.1	84.0	91.6	64.7	77.5	91.4

Discussions

► Limitations & (Runtime & Memory)

한계 (Limitations):

지원 불가: 어안(fisheye), 파노라마 이미지

성능 저하: 극단적 회전, 큰 비강체(non-rigid) 변형

장점: 구조 변경 없이 파인튜닝만으로도 쉽게 적응 가능

→ 기존 방식은 복잡한 최적화 재설계 필요

메모리 & 실행 시간 (Runtime & Memory):

평가 환경: NVIDIA H100 + Flash Attention v3

카메라 헤드: 전체 런타임의 5%, 메모리의 2% 수준

DPT 헤드: 프레임당 0.03초, 0.2GB

다중 프레임 처리: 메모리 여유 시 한 번에 가능, 부족 시 프레임별 추론 가능

해결책: LLM용 최적화 기법(예: Tensor Parallelism) 활용 가능

Input Frames	1	2	4	8	10	20	50	100	200
Time (s)	0.04	0.05	0.07	0.11	0.14	0.31	1.04	3.12	8.75
Mem. (GB)	1.88	2.07	2.45	3.23	3.63	5.58	11.41	21.15	40.63

Discussions

► Patchifying & Differentiable Bundle Adjustment

이미지 토큰화 (Patchifying):

이미지를 토큰화할 때 14×14 컨볼루션 레이어 및 DINOv2 사전학습 모델의 성능을 비교

→ 실험 결과, DINOv2가 더 나은 성능과 안정성 제공, 특히 학습 초기에 효과적

→ 하이퍼파라미터(학습률, 모멘텀) 변화에도 DINOv2가 덜 민감

→ 따라서 기본 토큰화 방식으로 DINOv2 채택

Differentiable Bundle Adjustment (BA):

VGGSfM에서 사용한 Differentiable BA를 시도한 결과,

소규모 실험에서는 유망한 성능을 보였으나,

→ 학습 시 연산 비용이 매우 높음 (PyTorch + Theseus 사용 시 학습 속도 4배 느림)

대규모 학습에는 비효율적이므로 본 논문에서는 제외

그러나 명시적 3D 주석이 없는 경우 유용한 지도 신호가 될 수 있어 향후 연구 방향으로 유망

Discussions

► Single-view Reconstruction & Normalizing Prediction

단일 이미지 재구성(Single-view Reconstruction):

기존 DUS3R, MAS3R는 한 이미지를 복제하여 쌍을 만들지만, VGGT는 단일 이미지 입력을 자연스럽게 지원
이 경우 global attention이 frame-wise attention으로 대체됨
명시적으로 단일 이미지 학습을 하지는 않았지만, 놀라운 재구성 성능을 보임



예측 정규화(Normalizing Prediction)

3D 포인트들의 평균 유클리드 거리로 GT 정규화 수행

DUS3R처럼 네트워크 출력까지 정규화하는 방식은 오히려 불안정하며,

학습 수렴에 불리하고 성능 향상에도 도움되지 않음을 발견

-> 따라서 정답만 정규화하고, 예측은 학습을 통해 모델이 자동으로 맞추도록 설계

Conclusion

VGGT (Visual Geometry Grounded Transformer) :

수백 장의 이미지를 입력받아 카메라 파라미터, 깊이, 포인트 클라우드, 포인트 트래킹 등 다양한 3D 정보를 직접 추정하는 **Feed-forward 신경망**

기존의 최적화 기반 방식과 달리, 단순하고 효율적인 신경망 중심(neural-first) 접근으로 후처리 없이도 최고 수준의 성능(SOTA)을 달성

이러한 구조는 실시간 애플리케이션에도 적합하며, 최적화 기반 방법보다 빠르고 간편하다